

NAMIBIA UNIVERSITY OF SCIENCE AND TECHNOLOGY

TrueCheck

By

Student Number	Name
222074078	Kuwunda Samundengu
223072397	Shamiso Vushe
223031461	Edna Alexandra
223023795	Mario Uushunga

A research article submitted in partial fulfillment of the requirements for the course

Natural Language Processing and Applications

Bachelor of Artificial Intelligence Honours
2026

Contents

1 Abstract **3**

2 Introduction **3**

3 Literature Review **4**
3.1 Introduction to the Review . . . 4
3.2 Benchmarking Claim Verification . . . 4
3.3 Contextual Representations for Text-Pair Classification . . . 4
3.4 First-Stage Evidence Retrieval . . . 4
3.5 Generating Explanations for Verdicts . . . 5
3.6 Comparative Synthesis . . . 5
3.7 Research Gaps and the Position of TrueCheck . . . 5
3.8 Summary of the Review . . . 6

4 Methodology **6**
4.1 Dataset Preparation . . . 6
4.1.1 Dataset Selection and Sources . . . 6
4.1.2 FEVER Cleaning and Dataset Splits . . . 7
4.1.3 Annotated Evidence Preparation . . . 7
4.1.4 External Datasets for Bias and Error Analysis . . . 7
4.2 Claim Classification . . . 7
4.2.1 Experimental Design . . . 7
4.2.2 Classical Model and Text Representation . . . 8
4.2.3 BERT Fine-Tuning and Input Handling . . . 8
4.2.4 Model Selection and Test Evaluation . . . 9
4.2.5 Bias and Error Evaluation . . . 9
4.3 Evidence Retrieval . . . 10
4.3.1 Evidence Corpus . . . 10
4.3.2 Passage Representation . . . 10
4.3.3 Retrieval Methods . . . 10
4.3.4 Index Construction . . . 11
4.3.5 Retriever Evaluation . . . 11
4.3.6 Evidence Sufficiency Analysis . . . 11
4.3.7 Choosing the Number of Passages . . . 12
4.3.8 Design Decisions from Testing . . . 12
4.3.9 Retrieval in the Application . . . 12
4.4 Explanation Generation . . . 12
4.4.1 Provider and Model Configuration . . . 12
4.4.2 Grounded Explanation and Citation Rules . . . 13
4.4.3 Output Validation and Failure Handling . . . 13
4.5 Web Application . . . 13
4.5.1 Architecture and Verification Flow . . . 13
4.5.2 Image, PDF and Speech Input . . . 14
4.5.3 Results, History and Exports . . . 14
4.5.4 Application Testing . . . 15
4.5.5 Project Repository . . . 15

5 Results and Discussion **15**
5.1 Dataset Preparation Results . . . 15
5.1.1 Dataset Cleaning Outcomes . . . 15
5.1.2 Final Dataset Distribution . . . 15
5.1.3 Evidence Preparation and Input Lengths . . . 16
5.2 Model Comparison and Final Classifier . . . 16
5.2.1 Training and Validation Results . . . 16
5.2.2 Comparison of the Six Experiments . . . 16
5.2.3 Final Classifier and Test Results . . . 17

5.3 Retrieval and Evidence-Count Results . . . 19
5.3.1 Retriever Comparison . . . 19
5.3.2 Evidence Sufficiency . . . 20
5.3.3 Number of Passages . . . 20
5.4 Bias and Error Analysis . . . 22
5.4.1 Dataset Selection and Evaluation Scope . . . 22
5.4.2 Cross-Domain Performance and Confusion Matrices . . . 23
5.4.3 Error Patterns and Selected Examples . . . 24
5.5 Application Testing and Explanation Examples . . . 24
5.5.1 Application Features and Integration Checks . . . 24
5.5.2 Multilingual Demonstrations . . . 26
5.6 Limitations . . . 27

6 Conclusion **27**

7 How We Used AI Tools **27**

8 References **29**

Appendix A Proposed Project Schedule and Features **30**
A.1 Proposed Project Schedule . . . 30
A.2 Proposed Features . . . 30

1. Abstract

The increasing volume of textual information and the ease of sharing it online have increased the demand for reliable fact verification. This study introduces TrueCheck, a prototype for verifying short factual claims. The system retrieves relevant Wikipedia passages and uses a BERT model fine-tuned on FEVER to classify claims as SUPPORTS or REFUTES based on the retrieved evidence. A language model then generates explanations grounded in the retrieved passages and supported by citations.

This study compares a classical TF-IDF and Logistic Regression approach with BERT and evaluates four retrievers: TF-IDF, BM25, sentence embeddings and a hybrid approach. Using annotated evidence, BERT achieved 94.55% accuracy on the prepared FEVER test split, compared with 79.23% for the classical model. The hybrid retriever achieved an 85.2% complete-evidence retrieval rate within the top five results on the validation claims.

Further evaluation examined the system's performance under retrieval based conditions using 200 general factual claims. BERT achieved 71.5% accuracy when provided with a single retrieved passage, with accuracy generally decreasing as additional passages were included. Cross-dataset evaluation yielded accuracies of 72.08% on ClimateFever and 75.00% on SciFact. These findings demonstrate the potential of combining evidence retrieval and transformer-based classification for automated fact verification while highlighting the challenges of evidence selection and reliable claim classification.

2. Introduction

False information is a growing problem that society continues to face. According to The Reuters Institute's *Digital News Report 2025*, 73% of respondents in Africa expressed concern about distinguishing real information from fake information online [Newman et al., 2025]. Similarly, an Afrobarometer survey found that 64% of respondents blamed social media users for spreading information they knew to be false, the same proportion who blamed politicians and political parties. Furthermore, 75% believed that social media makes people more likely to believe false information [Afrobarometer, 2022].

Namibia faces the same challenge. According to the Institute for Public Policy Research (IPPR), false claims circulated during the 2024 elections, particularly through AI-generated content and WhatsApp. One example was a fake video of the President collapsing at a rally [Institute for Public Policy Research, 2025]. This demonstrates how easily fabricated content can circulate and potentially influence public perception.

Verifying a factual claim often requires time and effort, which many people may not be willing or able to spend. Automated fact-checking research seeks to address this challenge by using computational methods to assess the accuracy of claims [Guo et al., 2022]. However, a system that provides only a classification label expects readers to trust it without understanding the reasoning behind it. A wrong label can create a false impression that a claim has been verified, potentially making the system misleading rather than useful. Therefore, an effective fact-checking tool should provide supporting evidence, explain its verdict based on that evidence, and avoid presenting uncertain predictions as verified facts.

This study introduces TrueCheck, a prototype designed to help users assess short factual claims by retrieving relevant evidence and providing an explanation of the classification. It combines evidence retrieval, machine learning, and language processing to produce results that users can examine rather than relying on prediction alone. This study also considers the challenges of applying automated fact-checking beyond the conditions in which a model was trained. This study addresses the following research questions:

1. How does a classifier trained on annotated evidence perform when provided with automatically retrieved evidence?
2. To what extent does the classifier's performance generalise to claims from other domains?

The remainder of this study is structured as follows. Chapter 3 reviews related work on automated fact-checking and retrieval. Chapter 4 describes the dataset, the classifiers, the retrievers and the application. Chapter 5 reports the results, including the retriever comparison, the evidence-count test and the bias and error analysis, and discusses their

limitations. Chapter 6 concludes, and Chapter 7 describes the use of AI tools.

3. Literature Review

3.1 Introduction to the Review

Automated claim verification involves finding evidence and deciding whether it supports or refutes a claim. This requires methods for retrieving relevant text, comparing claims with evidence and explaining the resulting predictions.

This chapter reviews research on claim-verification datasets, contextual text representations, evidence retrieval and explanation generation. It compares the approaches, discusses their limitations and explains their relevance to TrueCheck.

3.2 Benchmarking Claim Verification

Thorne et al. [2018] introduced FEVER, a dataset containing 185,445 claims created by modifying Wikipedia sentences. Claims were labelled as supported, refuted or not enough information. Supported and refuted claims also included annotated evidence sentences. This allowed systems to be assessed on both label prediction and evidence retrieval.

Their baseline achieved 31.87% when evaluation required a correct label and the necessary evidence, compared with 50.91% when evidence correctness was excluded from scoring. The second figure did not represent claim-only classification. The comparison showed that predicting a label and retrieving the evidence needed to justify it were separate challenges.

FEVER provided labelled claims and evidence references suitable for developing a verification pipeline. However, its claims were constructed from Wikipedia sentences, which limited how closely the benchmark represented naturally occurring misinformation.

Relevance to TrueCheck. FEVER provided the claims, classification labels and annotated evidence used for model development. Its evidence annotations also allowed retrieval results to be checked against known evidence sentences.

3.3 Contextual Representations for Text-Pair Classification

Devlin et al. [2019] introduced BERT, which learns contextual text representations using information from both sides of a word. Pre-training used masked language modelling and next-sentence prediction. The model could then be fine-tuned for tasks such as classification and natural language inference.

BERT's sentence-pair format was relevant to claim verification because the model could receive a claim and its evidence together. This allowed the classifier to learn relationships between the two texts. A TF-IDF baseline instead represented words and phrases as weighted features. Although these features could capture useful wording patterns, they did not explicitly model how the meaning of a claim related to its evidence.

However, strong classification scores did not necessarily demonstrate evidence-based reasoning. Schuster et al. [2019] found that FEVER claim-only classifiers could predict labels using wording patterns without considering evidence. Performance dropped on an evaluation set designed to reduce those patterns.

Relevance to TrueCheck. These studies supported comparing classical and BERT classifiers, as well as comparing claim-only and claim-and-evidence inputs. The comparisons could help assess whether evidence contributed useful information beyond patterns in claim wording.

3.4 First-Stage Evidence Retrieval

Karpukhin et al. [2020] studied dense passage retrieval for open-domain question answering. Their approach used separate encoders for questions and passages, allowing passages to be represented in advance and ranked through vector similarity. Their trained dense retriever improved top-20 retrieval accuracy over a strong BM25 baseline by 9 to 19 percentage points across the evaluated datasets.

Lexical and dense retrieval offered different advantages. Lexical methods matched words and terminology directly, while dense methods could retrieve related text when the query and passage used different wording. However, semantic similarity did not necessarily mean that a passage contained evidence supporting or contradicting a claim.

Cormack et al. [2009] introduced reciprocal rank fusion to combine results from multiple retrieval systems. The method used document positions in ranked lists rather than directly adding the systems' original scores. This provided a practical way to combine retrieval methods whose scores used different scales.

Relevance to TrueCheck. These studies supported comparing lexical and dense retrieval and testing a combined approach. However, TrueCheck did not reproduce the trained dense retriever used by Karpukhin et al. Its design used pretrained sentence embeddings and BM25, with their rankings combined through reciprocal rank fusion. Retrieval relevance and evidence sufficiency therefore needed to be assessed separately.

3.5 Generating Explanations for Verdicts

Atanasova et al. [2020] studied the automatic generation of explanations for fact-checking verdicts. Their approach selected sentences from detailed fact-checking reports to form shorter explanations. Experiments used LIAR-PLUS and DistilBERT-based models. Training explanation selection and verdict prediction together improved classification performance and the assessed quality of explanations compared with separate training.

The study showed the value of evaluating explanations directly rather than assessing only predicted labels. However, improved explanation quality did not establish that every explanation accurately represented the model's internal reasoning.

Gao et al. [2023] introduced ALCE, a benchmark for evaluating generated answers with citations. Evaluation covered fluency, correctness and citation quality. Their findings showed that even strong models could produce answers without complete citation support. A citation could therefore be present without fully supporting the associated statement.

Relevance to TrueCheck. These studies supported including explanations and checking them against the supplied evidence. TrueCheck generated explanations after BERT classification, rather than jointly training the two components. Its explanations could describe the evidence without necessarily revealing the reasoning used internally by BERT. Checking citation IDs alone could not establish that every generated statement was supported.

3.6 Comparative Synthesis

Table 3.1 summarises the reviewed studies and their relevance to TrueCheck.

Table 3.1: Reviewed studies and their relevance to TrueCheck

Study	Main contribution	Relevance to TrueCheck
Thorne et al. [2018]	FEVER claims, labels and annotated evidence	Dataset selection and evidence-based evaluation
Devlin et al. [2019]	Contextual representations suitable for sentence-pair tasks	Claim-and-evidence classification
Schuster et al. [2019]	Evidence that claim wording can provide shortcuts for label prediction	Comparison of claim-only and claim-and-evidence inputs
Karpukhin et al. [2020]	Trained dense retrieval for open-domain question answering	Comparison of lexical and semantic retrieval
Cormack et al. [2009]	Combining retrieval rankings through reciprocal rank fusion	Hybrid retrieval
Atanasova et al. [2020]	Joint explanation selection and verdict prediction	Explanation generation and evaluation
Gao et al. [2023]	Evaluation of correctness and citation support in generated answers	Grounding and citation checks
Diggelmann et al. [2020] and Wadden et al. [2020]	Verification datasets containing climate and scientific claims	External evaluation across different domains

The studies showed why classification, retrieval and explanation quality needed separate evaluation. A correct label could result from wording patterns, a highly ranked passage could lack the required evidence, and an explanation could contain citations without being fully supported. Evaluating only one component would therefore provide an incomplete assessment of a verification system.

3.7 Research Gaps and the Position of TrueCheck

The reviewed studies identified several challenges relevant to TrueCheck. These were practical evaluation concerns rather than problems that the wider literature had never addressed.

Domain differences. Diggelmann et al. [2020] introduced ClimateFever using real-world climate claims, extending the FEVER approach beyond artificially constructed claims. Wadden et al. [2020] introduced SciFact using scientific claims, research abstracts and annotated evidence sentences. The SciFact study found that domain adaptation improved performance compared with models trained on Wikipedia or political news. These findings supported evaluating TrueCheck's frozen classifier on external datasets.

Handling insufficient evidence. FEVER included NOT ENOUGH INFO as a separate label. This highlighted the need to distinguish unsupported claims from claims contradicted by evidence. For a binary classifier, handling missing evidence required additional application behaviour. Related passages alone could not establish that enough evidence had been retrieved.

Explanation grounding and faithfulness. An explanation needed to be supported by its cited passages. A separate question was whether it reflected the classifier's internal reasoning. TrueCheck's grounding checks addressed agreement with the supplied evidence, but did not establish that the explanation reproduced BERT's reasoning.

Multilingual processing. TrueCheck used an English evidence corpus and translated supported non-English claims before retrieval and classification. This introduced an additional concern because changes to names, numbers or negation could affect later stages. Translation quality and classifier performance therefore needed separate checks.

3.8 Summary of the Review

The reviewed research provided a basis for TrueCheck's dataset selection, classifier comparison, retrieval methods and explanation checks. It also highlighted the limits of relying on classification scores, retrieval rankings or visible citations alone. These findings motivated the project's two research questions concerning performance with retrieved evidence and performance across different domains.

4. Methodology

TrueCheck was developed using the design research methodology. This method addresses a practical problem by building an artefact and then testing how well that artefact solves the problem. It suited this project because the goal was a working system, and design options could be compared through testing. The artefact in this project is the TrueCheck system. The method had five phases: problem identification and motivation, objectives of the solution, design and development, evaluation, and communication. Table 4.1 shows how each phase was planned across the nine weeks of the project, from 7 September to 6 November 2026. The full proposed schedule and feature list are given in Appendix A. The phases overlapped, because each component reached them at a different time, and evaluation results often led back to design and development. The following sections describe dataset preparation, claim classification, evidence retrieval, explanation generation and the web application.

Table 4.1: Design research phases and planned timing

Phase	Planned weeks	Main work
1. Problem identification and motivation	Week 1	Understanding the project brief and the FEVER dataset
2. Objectives of the solution	Week 1	Agreeing how the system should work and which features to propose
3. Design and development	Weeks 1–8	Designing the system, then building the dataset, classifiers, retrievers, LLM component and web application
4. Evaluation	Weeks 3–9	Comparing classifiers, retrievers and evidence counts, testing the full system on new claims, and using the results to change the design
5. Communication	Weeks 4, 8 and 9	Presenting the project and submitting the report and code

4.1 Dataset Preparation

4.1.1 Dataset Selection and Sources

FEVER was selected because it provided labelled claims with Wikipedia evidence references. This supported comparisons between claim-only and claim-and-evidence classification. ClimateFever and SciFact provided external claims from different domains for bias and error analysis.

Table 4.2: Dataset sources and roles

Dataset	Data used	Purpose	Link
FEVER	Original training claims and supplied Wikipedia snapshot	Model development, evaluation and evidence corpus construction	View dataset
ClimateFever	Released claims and supplied Wikipedia evidence	Bias and error analysis of climate claims	View dataset
SciFact	Labelled development claims and scientific abstracts	Bias and error analysis of scientific claims	View dataset

4.1.2 FEVER Cleaning and Dataset Splits

Records were checked for unique numeric IDs, valid labels and nonempty claim text. Invalid claims were excluded, while invalid IDs or labels stopped preparation. Surrounding whitespace was removed without changing capitalisation, punctuation, numbers, negation or stop words.

Duplicate checks used exact text, normalised text and complete BERT token sequences. Confirmed duplicates retained the lowest-ID record and combined evidence annotations. Conflicting-label and questionable duplicate groups were excluded.

Cleaned records were divided into stratified 70% training, 15% validation and 15% test splits using seed 42. Overlap checks covered IDs, normalised claims, duplicate groups and token sequences. The binary dataset preserved these assignments.

The original FEVER records contained `id`, `claim`, `label`, `verifiable` and `evidence`. The three-class classification dataset kept `id`, `claim` and `label`, excluding evidence because NOT ENOUGH INFO claims lacked annotated verifying evidence. The binary classification dataset removed NOT ENOUGH INFO claims and kept `id`, `claim`, `evidence` and `label`, with evidence references resolved into text. Both datasets dropped `verifiable` because the class labels already indicated whether a claim was verifiable. The final project CSVs also retained `annotation_records`, `tokenizer_revision` and `evidence_sources` as metadata for traceability.

4.1.3 Annotated Evidence Preparation

Wikipedia evidence was resolved using page identifiers and sentence indices. Unique sentences from complete annotated evidence sets were combined and formatted as "Page title: Evidence sentence". Source references were retained separately.

Incomplete sets were logged. Records without a complete usable set were excluded from the binary dataset. NOT ENOUGH INFO claims received no invented evidence and were used only in the three-class claim-only experiments.

Input lengths were checked using the BERT tokenizer. Longer evidence was retained through chunking rather than truncated.

4.1.4 External Datasets for Bias and Error Analysis

ClimateFever and SciFact were prepared for bias and error analysis using the same claim-cleaning and duplicate-handling rules applied to FEVER. Labels were mapped as shown in Table 4.3.

Table 4.3: External label mapping

Dataset	Original label	Evaluation label
ClimateFever	SUPPORTS	SUPPORTS
ClimateFever	REFUTES	REFUTES
SciFact	SUPPORT	SUPPORTS
SciFact	CONTRADICT	REFUTES

ClimateFever excluded NOT ENOUGH INFO and DISPUTED claims but retained all supplied evidence for eligible records, regardless of sentence labels. SciFact used complete annotated rationale sets from its development split. Records without verifying evidence were excluded. No new external splits were created. Repeated evidence was removed, and lossless input encoding was checked before evaluation.

4.2 Claim Classification

4.2.1 Experimental Design

Six experiments compared classification tasks, input formats and model families. Each setup used a classical model based on TF-IDF with Logistic Regression and a fine-tuned `bert-base-uncased` model.

Table 4.4: Classification experiments

Setup	Labels	Input	Models
Three-class claim-only	SUPPORTS, REFUTES, NOT ENOUGH INFO	Claim text	Classical and BERT
Binary claim-only	SUPPORTS, REFUTES	Claim text	Classical and BERT
Binary claim-and-evidence	SUPPORTS, REFUTES	Claim text and annotated evidence	Classical and BERT

All binary experiments shared records and splits, allowing direct comparisons between models and input approaches. Three-class results were interpreted separately because the task and dataset membership differed.

Training used Python on Windows 11. Classical models used scikit-learn on an Intel Core 7 240H processor. BERT fine-tuning used PyTorch and Hugging Face Transformers with CUDA on an NVIDIA GeForce RTX 5060 Laptop GPU.

4.2.2 Classical Model and Text Representation

TF-IDF was selected as a simple baseline that represented useful words and phrases, including names and negation. Word2Vec could represent semantic similarity, but combining word vectors into a document representation would require an additional design choice. TF-IDF provided a straightforward comparison with BERT's contextual representations.

Claim-and-evidence models used separate claim and evidence vectorisers whose outputs were combined. Vocabulary and IDF values were fitted on training data only and balanced class weights addressed unequal class frequencies.

Table 4.5: Classical model settings

Setting	Value
Text representation	TF-IDF
Feature blocks	Claim only, or separate claim and evidence vectors
Word n-grams	Unigrams and bigrams
Maximum vocabulary	50,000 features per vectoriser
Minimum / maximum document frequency	2 / 1.0
Lowercase conversion	Enabled
Stop-word removal	None
Sublinear TF / smoothed IDF	Enabled
Feature normalisation	L2
Classifier	Logistic Regression
Regularisation / solver	L2 / `lbfgs`
Candidate C values	0.1, 1 and 10
Class weights	Balanced
Maximum iterations / tolerance	1,000 / 0.0001
Random seed	42
Selection criterion	Highest validation macro F1 score

C controlled regularisation strength, with larger values applying less regularisation. Ties favoured smaller C, and non-convergence stopped training.

4.2.3 BERT Fine-Tuning and Input Handling

The pretrained `bert-base-uncased` model was selected because its contextual representations could capture relationships between claim and evidence wording. It was fine-tuned for each experimental setup.

Claim-only models received claim text. Claim-and-evidence models received sequence pairs separated by special tokens. Each input was limited to 512 tokens, including special tokens.

Long evidence was packed into sentence-based chunks. Oversized sentences were divided into token segments, with the full claim repeated in every chunk. Claims exceeding the available token budget were rejected rather than truncated.

Chunk logits were averaged before calculating probabilities:

$$\mathbf{z}_{\text{claim}} = \frac{1}{m} \sum_{j=1}^{m} \mathbf{z}_j \tag{4.1}$$

Here, m is the number of chunks and $\mathbf{z}_j$ is the logit vector for chunk j. This produced one prediction and one class-weighted

loss per claim.

Table 4.6: BERT training settings

Setting	Value
Pretrained model	`bert-base-uncased`
Maximum input length	512 tokens, including special tokens
Evidence handling	Lossless sentence-based chunking
Chunk aggregation	Mean raw logits before softmax
Physical batch size / gradient accumulation	8 claims / 2 batches
Effective target batch size	16 claims
Maximum epochs / early-stopping patience	5 / 2
Optimiser / learning rate	AdamW / constant 0.00002
Weight decay	0.01, excluding biases and LayerNorm scales
AdamW betas / epsilon	$(0.9, 0.999)$ / 0.00000001
Gradient clipping	1.0
Loss	Class-weighted cross-entropy
Precision / padding	Float32 / dynamic
Gradient checkpointing	Enabled, non-reentrant
Maximum chunks per forward call	8, with all chunks processed
Random seed	42
Checkpoint selection	Highest validation macro F1 score

4.2.4 Model Selection and Test Evaluation

Parameters and checkpoints were selected using validation macro F1 score. BERT ties favoured the earlier epoch.

The six experiments were evaluated on the validation set to select the final setup and model. Test results were then reported for the classical and BERT models using the final binary claim-and-evidence setup. Both models used the same test split and were locked before evaluation.

Evaluation reported accuracy, per-class precision, recall and F1 score, macro F1 score and confusion matrices:

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}} \tag{4.2}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{4.3}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{4.4}$$

$$\text{F1 score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.5}$$

$$\text{Macro F1 score} = \frac{1}{K} \sum_{c=1}^{K} \text{F1 score}_c \tag{4.6}$$

Overall performance was measured using accuracy and macro F1 score. Precision, recall and F1 score were calculated for each class. TP, FP and FN represented true positives, false positives and false negatives for each class, while K represented the number of classes. All metric values were reported as percentages.

4.2.5 Bias and Error Evaluation

The frozen application BERT classifier was evaluated on prepared ClimateFever and SciFact records without retraining. Predictions used supplied evidence, with expected labels used for scoring.

False-support and false-refute rates were also calculated:

$$\text{False-support rate (\%)} = \frac{\text{REFUTES claims predicted as SUPPORTS}}{\text{Total REFUTES claims}} \times 100 \tag{4.7}$$

$$\text{False-refute rate (\%)} = \frac{\text{SUPPORTS claims predicted as REFUTES}}{\text{Total SUPPORTS claims}} \times 100 \tag{4.8}$$

False support meant a REFUTES claim was predicted as SUPPORTS. False refute meant a SUPPORTS claim was predicted as REFUTES.

Raw and row-normalised confusion matrices supported comparison across unequal class sizes. Selected errors were examined with their evidence and confidence scores as illustrative examples rather than estimates of error frequency.

4.3 Evidence Retrieval

The classifier was trained on claims that came with evidence, but a claim entered by a user has none. The retrieval component finds relevant Wikipedia sentences for a new claim, using only the claim text. Building it involved four decisions: what to search, how to search it, how many passages to use, and what to do when there is not enough evidence.

Each decision was made by building the options and testing them. The rest of this subsection follows the work in order: building the corpus, implementing and indexing the retrievers, evaluating them, and integrating the chosen retriever into the application.

4.3.1 Evidence Corpus

The FEVER Wikipedia dump has 5,416,537 pages. The brief asked for a small evidence corpus, so only part of the dump was used:

- **Cited pages:** all 12,450 pages used as evidence by the claims in the training, validation and test splits.
- **Distractor pages:** 10,000 random pages that no claim used, picked with seed 42.

The distractor pages make the search harder. Without them, almost every page would be relevant to some claim. Every sentence on each page was included, not only the evidence sentences, so the retrievers also had to skip unrelated sentences on the right page.

Pages used by test claims were included because the corpus is where evidence comes from, not training data. The retrievers were never trained on claims or labels, so this does not leak test answers.

The final corpus had 156,302 passages: 108,512 from cited pages and 47,790 from distractor pages.

4.3.2 Passage Representation

Each passage is a single Wikipedia sentence. Sentence-level passages were chosen for two reasons. Sentences were used because FEVER marks evidence at sentence level, which made it easy to check whether a retrieved passage was real evidence. Single sentences also let the application cite exact evidence.

Each passage stored the following fields:

- **Passage ID:** a sequential number that is also the passage's row in every index.
- **Page and sentence number:** used to match passages with FEVER annotations.
- **Text:** the sentence as it appears in the dump.
- **Rendered form:** the page title followed by the sentence, which is the text that was indexed.
- **Link:** the Wikipedia address of the page, shown to users with the evidence.
- **Distractor flag:** whether the passage came from a distractor page.

The page title was added in front of each sentence, as in "Kenya: Its capital and largest city is Nairobi." Many sentences use words like "it" or "its" instead of naming the subject, so the title tells the retriever what the sentence is about. This is also the format BERT was trained on, so retrieved passages reach the classifier in a familiar form.

4.3.3 Retrieval Methods

Four retrievers were compared, covering lexical, dense and combined approaches. Lexical methods match the exact words of a claim, which suits the names and titles that are common in FEVER claims. Dense methods compare meaning, so they can find evidence when the claim and passage use different wording. All four indexed the rendered passages. Table 4.7 summarises their configuration.

Table 4.7: Retriever configurations

Retriever	Representation	Index or library	Ranking score
TF-IDF	Sparse unigrams and bigrams	scikit-learn	Cosine similarity
BM25	Sparse tokens	`bm25s`	BM25 score
Embeddings	Dense, 384 dimensions	FAISS flat index	Cosine similarity
Hybrid	BM25 and embeddings	Both of the above	Reciprocal rank fusion

TF-IDF Retriever

TF-IDF gives more weight to words that are rare across the corpus, such as names, and less to common words. It used single words and two-word phrases, removed English stop words, and ignored words found in more than half of all passages. Each claim was compared with every passage using cosine similarity, and passages that shared no words with the claim were never returned.

BM25 Retriever

BM25 also matches words, but it improves on TF-IDF in two ways. A word repeated many times in a passage adds less and less to its score, and long passages do not get an unfair advantage. It was built with the `bm25s` library using its default settings, with English stop words removed.

Embedding Retriever

The embedding retriever turns each passage and claim into a list of 384 numbers that represents its meaning, using the pretrained `all-MiniLM-L6-v2` model without further training. Passages with similar meaning to the claim are found using cosine similarity. The vectors were stored in a FAISS index that compares the claim with every passage, so the search is exact rather than approximate. This was fast enough at this corpus size.

Hybrid Retriever

The hybrid retriever combines BM25 and the embedding retriever, so that it can match both exact words and meaning. BM25 was used instead of TF-IDF because it found more evidence when the single retrievers were compared.

For each claim, it takes the top 50 passages from each retriever and gives every passage points based on its rank in each list, using reciprocal rank fusion. A higher rank earns more points, and a passage found by both retrievers earns points from both. The passages with the most points are returned. Ranks are used instead of raw scores because BM25 scores and cosine similarities are on different scales and cannot be added together.

Rank points only show position, not how relevant a passage is. The cosine similarity between the claim and each returned passage was therefore also kept as a separate relevance score.

4.3.4 Index Construction

Each retriever's index was built once from the same passage file and saved. Because every index uses the same passage order, results from different retrievers refer to the same passages and can be combined directly. The indexes were not rebuilt during testing or use.

4.3.5 Retriever Evaluation

The retrievers were compared on the validation split, using 2,000 claims with evidence and 2,000 NOT ENOUGH INFO claims. Test claims were not used. Each retriever received only the claim text.

The following measures were used:

- **Recall@k:** the percentage of claims whose complete evidence was found in the top k passages, for k of 1, 5, 10 and 50. A claim only counted as found if every sentence of its evidence was retrieved, because the classifier needs all of it.
- **Page@5:** the percentage of claims with an evidence page in the top five passages.
- **Mean reciprocal rank (MRR):** how high up the list the evidence appeared, on average.
- **Search time:** milliseconds per claim.

Recall and MRR mattered most, because evidence that is not retrieved cannot be used by the classifier. Search time was a secondary consideration.

4.3.6 Evidence Sufficiency Analysis

The classifier can only answer SUPPORTS or REFUTES, so it cannot say when there is not enough evidence. The project tested whether retrieval scores could show this instead. The idea was simple: if the best passage found for a claim has a low score, the corpus may have nothing relevant, and the system should say there is not enough evidence rather than give a verdict.

To test this, each claim was given a retrieval score. For the hybrid retriever, cosine similarity was used instead of rank points, because rank points do not show how relevant a passage is. The scores of claims with evidence were then compared with the scores of NOT ENOUGH INFO claims, using two measures.

- **ROC-AUC:** this measures how well a score separates two groups. It is the chance that a randomly chosen claim with evidence gets a higher score than a randomly chosen NOT ENOUGH INFO claim. A value of 1.0 means the

score separates the groups perfectly, and 0.5 means it does no better than guessing. A ROC curve was also plotted for each retriever to show this visually.

- **Threshold sweep:** this shows what would happen in practice if a score cutoff were used. Cutoffs were set to keep 80%, 85%, 90%, 95% and 99% of claims with evidence, and for each one the percentage of NOT ENOUGH INFO claims it would block was recorded. A useful cutoff would block many NOT ENOUGH INFO claims while keeping almost all claims with evidence.

4.3.7 Choosing the Number of Passages

The number of passages given to the classifier, k, matters. More passages can include more evidence, but also more irrelevant text. To choose k, a set of 200 general claims was written to test the retrieval and classification pipeline.

The set covered ten topics, with 20 claims each and 100 SUPPORTS and 100 REFUTES claims. The claims came in 100 true and false pairs, such as "The capital of Canada is Ottawa" and "The capital of Canada is Toronto". Each claim had a source, and its label was set before testing. The pairs were split into a selection set and a check set of 100 claims each, balanced by topic and label.

Each claim was retrieved once with the hybrid retriever, which returned its top ten passages. For each k from 1 to 10, the first k of those passages were given to BERT, after repeated passages were removed, in the same way as in the application. This gave 2,000 predictions. The value of k was chosen using the selection set only, by highest macro F1 score, with ties broken by accuracy and then by the smaller k. It was fixed before the check set predictions were computed, so the check set gives a separate measure of the chosen k. Repeating the run gave the same predictions.

Accuracy, macro F1 score and recall for each class were reported, along with results for each topic. Afterwards, the saved predictions were also used to test whether differences between values of k were statistically significant, and to give a confidence range for the check set accuracy. This set is a diagnostic rather than an independent benchmark, because some claims were added after earlier results had been seen. All claims were in English, and the LLM steps were not included.

4.3.8 Design Decisions from Testing

Testing led to three decisions. The results behind them are presented in Section 5.

- **Retriever:** the hybrid retriever was chosen because it achieved the highest recall and MRR.
- **Number of passages:** the default was changed from five passages to one, because one passage gave the highest macro F1 score on the selection set, and accuracy was generally lower with more passages.
- **Insufficient evidence:** no score cutoff was added, because retrieval scores separated claims with and without evidence only weakly.

4.3.9 Retrieval in the Application

In the application, the hybrid retriever runs locally on the saved indexes and returns one passage by default. Users can choose up to 10. Portuguese and Afrikaans claims are translated into English first, because the corpus is in English. At start-up, the application checks that all index files exist and match the corpus, and it will not start if they do not.

Retrieved passages are labelled E1, E2 and so on, and each keeps its page, Wikipedia link and scores. The same labels are used by the classifier, the explanation and the interface, so every cited passage can be traced back to its source.

If no passages are found, the application gives no prediction. Because no score cutoff is used, weakly related passages can still reach BERT, which always answers SUPPORTS or REFUTES. In those cases, the lack of evidence can only be pointed out in the explanation, and the interface warns users that this has not been calibrated.

4.4 Explanation Generation

4.4.1 Provider and Model Configuration

OpenAI's `gpt-6-luna` model was accessed through the API for claim preparation and explanation generation. Both stages used the same configuration saved in the application settings. The API model handled language processing and explanations, while the locally fine-tuned BERT classifier produced the SUPPORTS or REFUTES prediction.

Table 4.8: Model used for claim preparation and explanations

Provider	Model	Purpose
OpenAI	`gpt-6-luna`	Claim screening, language preparation, explanations and evidence translation

4.4.2 Grounded Explanation and Citation Rules

The preparation stage checked whether the input was a factual claim in English, Portuguese or Afrikaans. Questions, opinions, instructions, greetings, creative requests, gibberish and unsupported languages were rejected. False factual claims were accepted because screening assessed the input type rather than its truth. Accepted Portuguese and Afrikaans claims were translated into English, while English claims were preserved.

After retrieval and classification, the explanation model received the original claim, BERT prediction and retrieved passages labelled E1, E2 and so on. Instructions required explanations to use only these passages, cite the supplied IDs and identify missing or conflicting information. The model was instructed not to invent support for BERT or replace its prediction with a different verdict.

The prompt instructed the model to write explanations in two to four sentences, using no more than 150 words. For Portuguese and Afrikaans, it also required translations of every supplied passage, with source IDs linking each translation to the original English evidence.

4.4.3 Output Validation and Failure Handling

Responses used structured JSON containing screening outcomes, explanation text and citations. Multilingual outputs also required evidence translations. Validation checked required fields, known citation IDs and complete translation coverage. These checks verified response structure and source references rather than proving that every statement was supported.

Each stage had a 60-second time limit, including at most one repair attempt or retry after a temporary server error. If screening or output validation failed, no prediction was displayed. Provider errors produced a message explaining that the service was unavailable.

4.5 Web Application

4.5.1 Architecture and Verification Flow

TrueCheck was implemented as a locally hosted web application using a FastAPI backend and an HTML, CSS and JavaScript interface. Uvicorn served the application through a browser. Retrieval and BERT inference ran locally, while language processing, explanations, document extraction and speech transcription used external APIs.

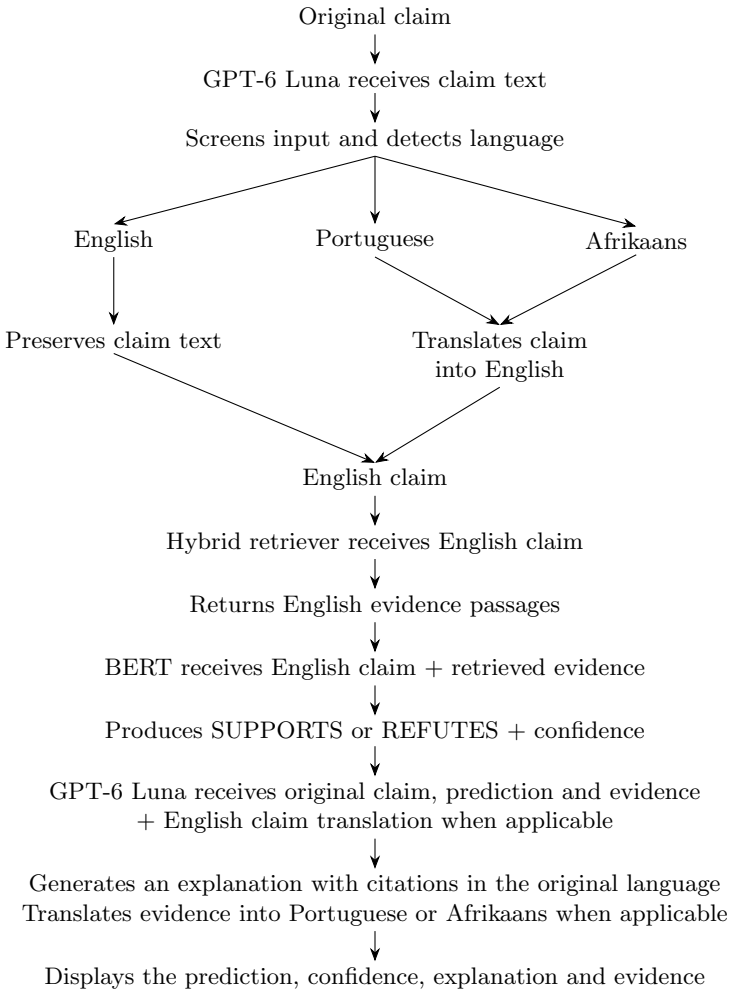


Figure 4.1: Application verification flow for accepted claims with retrieved evidence.

Rejected input stopped before retrieval. If no passages were retrieved, BERT classification and explanation generation were skipped, and no prediction was displayed.

4.5.2 Image, PDF and Speech Input

Images and PDFs were transcribed using Anthropic's `claude-sonnet-5-5` model. Voice recordings were transcribed using OpenAI's `gpt-transcribe` model. Extracted text and speech transcripts were available for review and editing before submission. Uploads were not added to the evidence corpus.

Read aloud used browser/device speech synthesis rather than the transcription API.

Table 4.9: Application input limits

Input	Limit
Claim text	4,000 characters, subject to BERT's token limit
Image/PDF upload	10 MiB
PDF length	10 pages
Extracted text	20,000 characters
Voice recording	30 seconds and 5 MiB
Retrieved passages	1–10, default 1

4.5.3 Results, History and Exports

Results displayed the model label, confidence, explanation, citations and retrieved evidence. A notice explained that confidence was not a guarantee of accuracy. Conversations were stored in browser local storage and could be reopened without repeating verification. PDF and Word reports were generated locally.

The provider and model could be configured separately for explanations, image/PDF extraction and speech transcription. Explanations and extraction supported OpenAI or Anthropic, while transcription supported OpenAI only. The reported demonstrations used `gpt-6-luna` for claim preparation and explanations, `claude-sonnet-5-5` for image/PDF extraction and `gpt-transcribe` for speech transcription.

Saved API credentials used Windows Credential Manager rather than browser history. Cancellation stopped the interface from accepting late responses.

4.5.4 Application Testing

Testing covered submission, uploads, transcription, history, exports, cancellation and provider failures. Mocked responses checked interface behaviour and error handling. Actual application examples checked English explanation grounding, rejection of a creative request and Portuguese/Afrikaans processing. These demonstrations were reported separately from aggregate classifier evaluation.

4.5.5 Project Repository

The source code, fine-tuned BERT model download and setup instructions are available in the project repository.

Click here to open the TrueCheck repository

The README provides separate guides for running the web application and recreating dataset preparation, model training and evaluation.

5. Results and Discussion

5.1 Dataset Preparation Results

5.1.1 Dataset Cleaning Outcomes

The original FEVER training dataset contained 145,449 claims across three classes: SUPPORTS, REFUTES and NOT ENOUGH INFO. Cleaning removed 8,090 duplicate copies and excluded 2,559 records from groups with conflicting labels. Another 71 records were excluded as questionable duplicates. These claims matched after normalisation or tokenisation using the BERT tokenizer, but it was unclear whether they were duplicates. This left 134,729 claims across the three classes, as shown in Table 5.1.

Table 5.1: Dataset cleaning outcomes

Outcome	Records
Original FEVER training dataset	145,449
Duplicate copies removed	8,090
Records excluded from conflicting-label groups	2,559
Records excluded from questionable duplicate groups	71
Retained across all three classes	**134,729**

5.1.2 Final Dataset Distribution

The cleaned dataset contained 134,729 claims and was used for the three-class experiments. The dataset was divided into 70% training, 15% validation and 15% test data using stratified splitting with seed 42. This kept the class proportions similar across the three splits.

Table 5.2: Final three-class dataset distribution

Split	SUPPORTS	REFUTES	NOT ENOUGH INFO	Total
Training	51,486	19,549	23,275	94,310
Validation	11,033	4,189	4,987	20,209
Test	11,033	4,189	4,988	20,210
Total	**73,552**	**27,927**	**33,250**	**134,729**

Removing the 33,250 NOT ENOUGH INFO claims left the final binary dataset of 101,479 claims labelled SUPPORTS or REFUTES. The binary dataset kept the existing split assignments. The same binary claims and splits were used for the four binary experiments and the final project's classical and BERT models. This kept the data consistent across the model comparisons and final evaluation.

Table 5.3: Final binary dataset distribution

Split	SUPPORTS	REFUTES	Total
Training	51,486	19,549	71,035
Validation	11,033	4,189	15,222
Test	11,033	4,189	15,222
Total	**73,552**	**27,927**	**101,479**

SUPPORTS claims made up about 72.5% of the binary dataset, while REFUTES claims made up 27.5%. Because of this class imbalance, accuracy alone was not enough to assess performance. Macro F1 score and per-class precision, recall and F1 score were also reported to show how well the models performed on both classes.

5.1.3 Evidence Preparation and Input Lengths

Evidence preparation kept all unique sentences from complete annotated evidence sets. Of the 101,479 claim-and-evidence pairs, 100,386 (98.92%) fitted within BERT's 512-token input limit. The remaining 1,093 pairs needed more than one chunk. Longer evidence was split into chunks without dropping any evidence tokens. These figures describe the annotated dataset. Claims entered into TrueCheck may need more than one chunk, depending on the length of the claim and retrieved evidence.

5.2 Model Comparison and Final Classifier

Six experiments helped determine the classification task and input format for TrueCheck. Classical (TF-IDF with Logistic Regression) and BERT (fine-tuned `bert-base-uncased`) models were compared across three setups. Validation results guided selection, while test results were used for final reporting.

Three-class classification used the labels SUPPORTS, REFUTES and NOT ENOUGH INFO, while binary classification used only SUPPORTS and REFUTES. Claim-only models received only the claim text, while claim-and-evidence models received the claim text together with annotated evidence.

5.2.1 Training and Validation Results

The classical models selected $C = 1$ for the three-class claim-only and binary claim-only setups, and $C = 10$ for the binary claim-and-evidence setup, based on validation macro F1 score. C controls regularisation strength, with larger values applying less regularisation. All candidates converged successfully.

Based on validation macro F1 score, the best BERT checkpoints were epoch 3 for the three-class claim-only setup, epoch 4 for the binary claim-only setup and epoch 2 for the binary claim-and-evidence setup. The binary claim-and-evidence BERT model stopped after epoch 4 following two epochs without validation improvement. Training loss continued to decrease, showing that lower training loss did not always lead to better validation performance.

5.2.2 Comparison of the Six Experiments

Tables 5.4 to 5.6 show the validation results for the three experimental setups. Bold values indicate the higher score in each row.

Table 5.4: Three-class claim-only validation results

Class	Metric	Classical	BERT
SUPPORTS	Precision	71.83%	**78.04%**
SUPPORTS	Recall	59.04%	**79.80%**
SUPPORTS	F1 score	64.81%	**78.91%**
REFUTES	Precision	54.84%	**68.66%**
REFUTES	Recall	**63.31%**	62.97%
REFUTES	F1 score	58.77%	**65.70%**
NOT ENOUGH INFO	Precision	41.37%	**56.28%**
NOT ENOUGH INFO	Recall	52.30%	**57.39%**
NOT ENOUGH INFO	F1 score	46.20%	**56.83%**
Overall	Accuracy	58.26%	**70.78%**
Overall	Macro F1 score	56.59%	**67.14%**

BERT achieved higher accuracy and a higher macro F1 score than the classical model. NOT ENOUGH INFO had the lowest F1 score for both models. This supported the decision to keep the final classifier binary and address insufficient evidence at the application level.

Table 5.5: Binary claim-only validation results

Class	Metric	Classical	BERT
SUPPORTS	Precision	87.27%	**88.76%**
SUPPORTS	Recall	84.08%	**92.60%**
SUPPORTS	F1 score	85.64%	**90.64%**
REFUTES	Precision	61.75%	**78.01%**
REFUTES	Recall	67.70%	**69.11%**
REFUTES	F1 score	64.59%	**73.29%**
Overall	Accuracy	79.57%	**86.14%**
Overall	Macro F1 score	75.11%	**81.97%**

BERT achieved higher scores across all metrics in the binary claim-only task. Macro F1 score reached 81.97%, compared with 75.11% for the classical model. Both models performed better on SUPPORTS than REFUTES.

Table 5.6: Binary claim-and-evidence validation results

Class	Metric	Classical	BERT
SUPPORTS	Precision	87.05%	**96.08%**
SUPPORTS	Recall	83.49%	**96.73%**
SUPPORTS	F1 score	85.23%	**96.40%**
REFUTES	Precision	60.75%	**91.23%**
REFUTES	Recall	67.27%	**89.62%**
REFUTES	F1 score	63.84%	**90.41%**
Overall	Accuracy	79.03%	**94.77%**
Overall	Macro F1 score	74.54%	**93.41%**

Adding evidence increased BERT's macro F1 score from 81.97% to 93.41%. REFUTES recall also increased from 69.11% to 89.62%. The classical model showed a small decrease in overall performance.

The shared binary data allowed a direct comparison of the input approaches, with BERT performing better when evidence was included. NOT ENOUGH INFO scores remained too low for the intended use. The three-class models used only claim text because FEVER did not provide annotated evidence for NOT ENOUGH INFO claims. These findings supported binary claim-and-evidence input with BERT, while insufficient evidence was addressed separately in the application.

5.2.3 Final Classifier and Test Results

The binary claim-and-evidence input approach and BERT model were chosen for the TrueCheck application based on validation results. Both final models were then evaluated on the same 15,222 test claims, consisting of 11,033 SUPPORTS claims and 4,189 REFUTES claims.

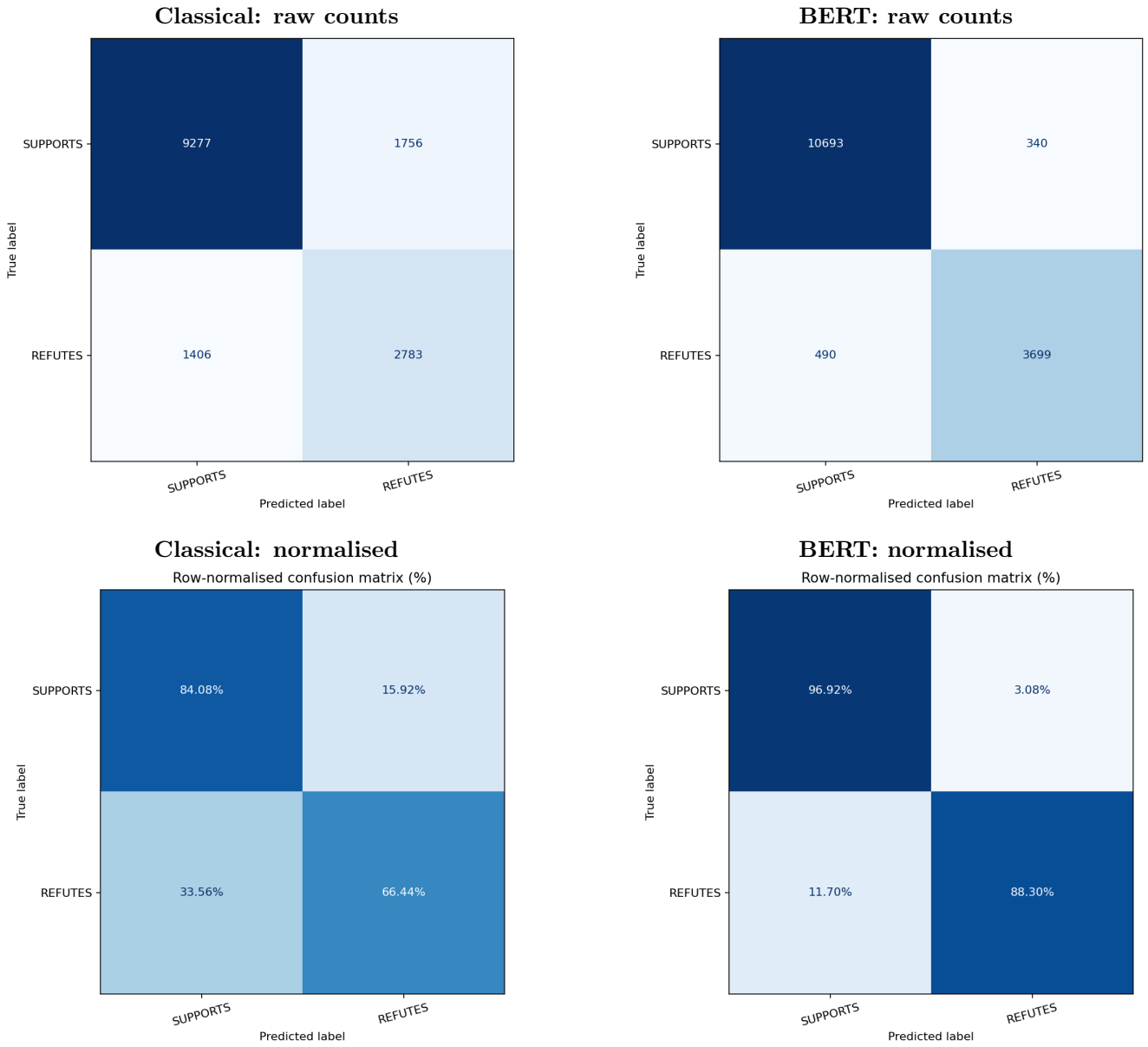


Figure 5.1: Raw-count and row-normalised confusion matrices for the final classical and BERT models on the test set.

Figure 5.1 shows that BERT made 830 errors, compared with 3,162 for the classical model. The classical model misclassified 1,756 SUPPORTS claims and 1,406 REFUTES claims, compared with 340 and 490 for BERT. Both models had higher error rates on REFUTES, at 33.56% for the classical model and 11.70% for BERT, compared with 15.92% and 3.08% on SUPPORTS.

Table 5.7: Final test results

Class	Metric	Classical	BERT
SUPPORTS	Precision	86.84%	**95.62%**
SUPPORTS	Recall	84.08%	**96.92%**
SUPPORTS	F1 score	85.44%	**96.26%**
REFUTES	Precision	61.31%	**91.58%**
REFUTES	Recall	66.44%	**88.30%**
REFUTES	F1 score	63.77%	**89.91%**
Overall	Accuracy	79.23%	**94.55%**
Overall	Macro F1 score	74.61%	**93.09%**

Table 5.7 summarises the overall and per-class test results. BERT achieved higher scores across all metrics, supporting the earlier decision to use BERT in TrueCheck. These results used annotated FEVER evidence rather than passages retrieved by the application.

5.3 Retrieval and Evidence-Count Results

This section presents the results of the three retrieval tests described in the methodology: the retriever comparison, the evidence sufficiency analysis and the choice of the number of passages.

5.3.1 Retriever Comparison

Table 5.8 compares the four retrievers on the 2,000 validation claims with evidence, searching all 156,302 passages. Bold values show the best result in each column.

Table 5.8: Retriever comparison on 2,000 validation claims with evidence

Retriever	R@1	R@5	R@10	R@50	Page@5	MRR	ms/claim
TF-IDF	43.9%	72.1%	82.0%	93.3%	93.8%	0.627	**0.92**
BM25	55.1%	79.2%	85.9%	93.6%	96.0%	0.724	3.45
Embeddings	58.8%	83.8%	88.8%	93.6%	98.0%	0.767	1.94
Hybrid	**60.2%**	**85.2%**	**90.1%**	**96.0%**	**98.8%**	**0.777**	7.37

The hybrid retriever performed best on every recall measure, on Page@5 and on MRR. With five passages, it found the complete evidence for 85.2% of claims.

Embeddings did better than the two word-matching methods. At five passages, embeddings found complete evidence for 83.8% of claims, compared with 79.2% for BM25 and 72.1% for TF-IDF. This suggests that matching meaning helps when a claim is worded differently from its evidence.

Combining BM25 and embeddings helped most when more passages were retrieved. The hybrid retriever was 1.4 percentage points ahead of embeddings at five passages and 2.4 points ahead at 50. At 50 passages, BM25 and embeddings each reached 93.6%, but the hybrid reached 96.0%. This means the two methods often found evidence for different claims, so combining them covered more claims than either one alone. Figure 5.2 shows this pattern.

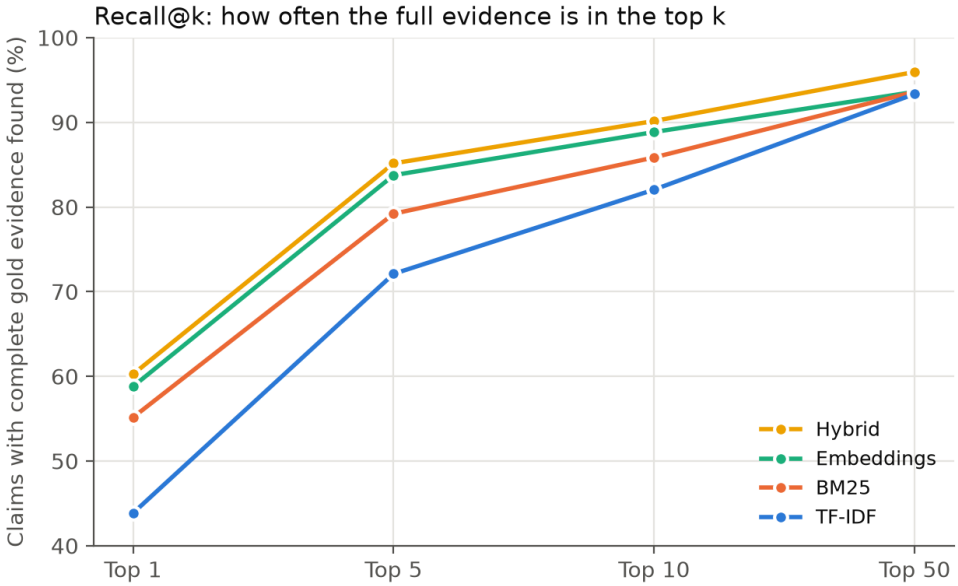


Figure 5.2: Recall@k for the four retrievers on validation claims.

The hybrid retriever was the slowest, at 7.37 ms per claim, because it runs two searches. This is still very fast compared with classification and explanation, so the hybrid retriever was chosen for TrueCheck. Its lead over embeddings was small with few passages, and these differences were not tested for statistical significance.

Two further points stand out. First, the hybrid found complete evidence in its top passage for only 60.2% of claims. A single sentence can only contain the full evidence for claims that need just one sentence, so no retriever could reach 100% with one passage. Second, Page@5 was much higher than Recall@5, at 98.8% compared with 85.2% for the hybrid. The retrievers usually found the right page, and most misses came from picking the wrong sentences on that page.

These results are probably better than TrueCheck would achieve on the whole of Wikipedia. The corpus contains every page used as evidence, and only 47,790 of its passages come from distractor pages. A search of all of Wikipedia would face far more competing passages.

5.3.2 Evidence Sufficiency

Retrieval scores were poor at telling claims with evidence apart from NOT ENOUGH INFO claims. ROC-AUC was 0.568 for TF-IDF, 0.560 for BM25, 0.654 for embeddings and 0.655 for the hybrid. A value of 0.5 means no better than guessing, so even the best retrievers were only slightly better than chance (Figure 5.3).

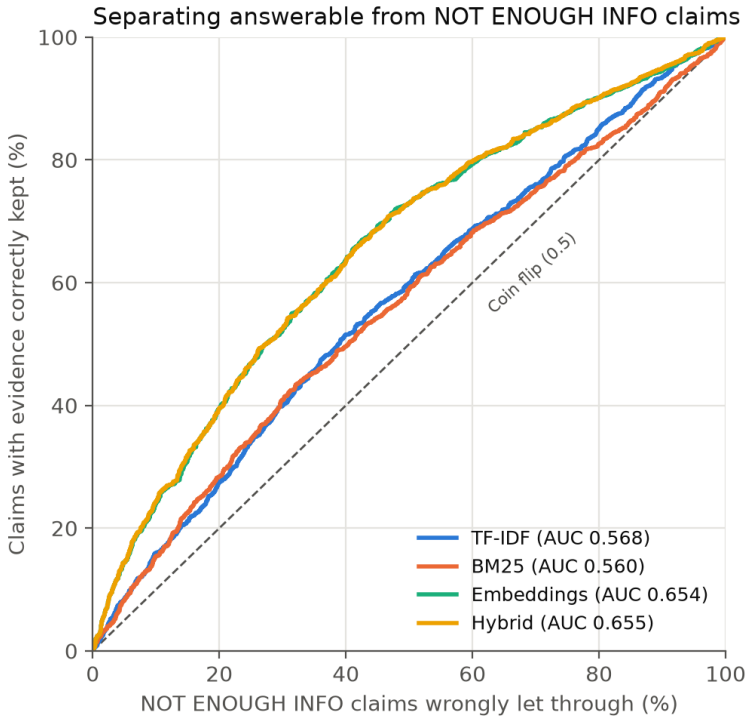


Figure 5.3: ROC curves for separating claims with evidence from NOT ENOUGH INFO claims.

Table 5.9 shows what would happen if a score cutoff were used. Each row sets the cutoff so that a given share of claims with evidence is kept, and shows how many NOT ENOUGH INFO claims would be blocked.

Table 5.9: NOT ENOUGH INFO claims blocked at each keep rate for claims with evidence

Keep rate	TF-IDF	BM25	Embeddings	Hybrid	Hybrid cutoff
80%	25.6%	23.8%	39.1%	39.6%	0.6632
85%	20.1%	16.6%	30.1%	30.0%	0.6389
90%	14.2%	10.9%	20.8%	20.3%	0.6078
95%	8.5%	5.7%	9.1%	9.9%	0.5629
99%	1.7%	1.1%	2.5%	2.8%	0.4983

No cutoff gave a good trade-off. With the hybrid retriever, keeping 95% of claims with evidence blocked only 9.9% of NOT ENOUGH INFO claims. Blocking 39.6% of them meant also blocking 20% of claims that did have evidence.

A likely reason is that a similarity score shows whether a passage is about the same topic as the claim, not whether it proves or disproves it. A NOT ENOUGH INFO claim about a well-known subject can still find closely related passages and get a high score. A score cutoff on its own is therefore not a reliable way to detect missing evidence, and none was added to the application.

5.3.3 Number of Passages

Table 5.10 shows the retrieval and classification results for each number of passages, k, on the 200 general claims. Figure 5.4 plots accuracy and macro F1 score.

One passage gave the best results. On the selection set, $k = 1$ had the highest macro F1 score, at 68.85%, so it was chosen. On the separate check set, $k = 1$ reached 74.00% accuracy and a macro F1 score of 73.83%, again the highest of any k. With only 100 check claims, the true accuracy is likely to lie between 64.6% and 81.6% (95% Wilson interval). The application's default was therefore changed from five passages to one.

More passages made results worse. Across all 200 claims, accuracy fell from 71.50% with one passage to 61.50% with ten. A McNemar test on the paired predictions showed that one passage was significantly better than five ($p = 0.005$) and ten ($p = 0.003$). The difference between one and two passages was not significant ($p = 0.19$). On the check set alone, with fewer claims, the results were $p = 0.057$ against five passages and $p = 0.019$ against ten. These tests were not corrected for making several comparisons, so they should be read as a guide.

Table 5.10: Retrieval and classification results by number of retrieved passages

	Selection (100)		Check (100)		All (200)		
k	Accuracy	Macro F1	Accuracy	Macro F1	Accuracy	SUPPORTS recall	REFUTES recall
1	**69.00%**	**68.85%**	**74.00%**	**73.83%**	**71.50%**	71.00%	**72.00%**
2	64.00%	63.64%	72.00%	71.90%	68.00%	70.00%	66.00%
3	59.00%	58.67%	72.00%	71.99%	65.50%	69.00%	62.00%
4	59.00%	58.50%	71.00%	71.00%	65.00%	70.00%	60.00%
5	59.00%	58.50%	66.00%	65.99%	62.50%	69.00%	56.00%
6	59.00%	58.50%	67.00%	66.97%	63.00%	70.00%	56.00%
7	60.00%	59.20%	65.00%	65.00%	62.50%	70.00%	55.00%
8	62.00%	61.24%	65.00%	64.97%	63.50%	**72.00%**	55.00%
9	59.00%	58.06%	64.00%	63.94%	61.50%	71.00%	52.00%
10	60.00%	59.20%	63.00%	62.97%	61.50%	70.00%	53.00%



Figure 5.4: Accuracy and macro F1 score by number of retrieved passages.

Almost all of the drop came from false claims. REFUTES recall fell from 72% to 53%, while SUPPORTS recall stayed between 69% and 72%. As more passages were added, BERT predicted SUPPORTS more often, rising from 99 of the 200 claims with one passage to 117 with ten. For example, “The capital of Kenya is Mombasa” was correctly labelled REFUTES with one passage (confidence 98.9%), which said that Nairobi is the capital. With five passages, the evidence also included sentences mentioning Mombasa, and BERT changed its answer to SUPPORTS (confidence 99.3%).

Input length did not cause this drop, because every input fitted within BERT’s 512-token limit even with ten passages. A more likely cause, which was not tested directly, is the difference between training and use. BERT was trained on annotated evidence, and 88.2% of training claims had complete evidence in a single sentence. Extra retrieved passages are on the same topic but are not real evidence, and a passage that shares words with a false claim can look like support.

The retrieval and classification diagnostic produced lower scores than the evaluation using annotated evidence, which achieved a test macro F1 score of 93.09% in Table 5.7. These evaluations used different claims, so the performance gap could not be attributed to retrieval alone. Cases of unrelated retrieval were also observed. For 29 of the 200 claims, the top passage came from a distractor page. For example, “Mount Fuji is located in South Korea” retrieved a passage about Fuji Television. Accuracy with one passage also varied by topic, from 55% for Animals and for Arts and music to 90% for History and exploration and for Books and authors, although each topic had only 20 claims.

BERT was often confident even when it was wrong. Errors with a confidence of at least 99% rose from 19 with one passage to 29 with ten. BERT’s confidence should therefore not be read as the chance that a claim is true.

Using only one passage also has a cost. One sentence cannot hold the full evidence for a claim that needs several sentences, and the explanation can only cite that one passage. Users can choose more passages when they want more context, although Table 5.10 suggests that the label may then be less reliable.

In this test, retrieval took an average of 180.7 ms per claim, and classification took 67.6 ms with one passage and 149.7 ms with ten, on an NVIDIA GeForce RTX 5060 Laptop GPU. Retrieval was slower here than in Table 5.8 because claims were searched one at a time, as in the application.

5.4 Bias and Error Analysis

5.4.1 Dataset Selection and Evaluation Scope

The analysis focused on ClimateFever and SciFact to compare performance across climate and scientific claims. Both supplied labelled claims with evidence, allowing the frozen BERT classifier to be evaluated without retraining.

Both datasets used the same claim-cleaning and duplicate-handling rules applied to FEVER. Claim wording was preserved apart from removing surrounding whitespace. Duplicate checks used exact text, normalised text and BERT token sequences. Confirmed duplicates were merged, while conflicting-label and questionable duplicate groups were excluded.

Table 5.11: External dataset preparation and label mapping

Preparation detail	ClimateFever	SciFact
Original records	**1,535 total:** 654 SUPPORTS, 253 REFUTES, 474 NOT ENOUGH INFO, 154 DISPUTED	**300 total:** 124 SUPPORT, 64 CONTRADICT, 112 without annotated evidence
SUPPORTS mapping	SUPPORTS → SUPPORTS	SUPPORT → SUPPORTS
REFUTES mapping	REFUTES → REFUTES	CONTRADICT → REFUTES
Evidence used	All supplied evidence sentences associated with each retained record	All sentences from complete annotated rationale sets associated with each retained record
Exclusions	1 duplicate copy, 474 NOT ENOUGH INFO claims and 154 DISPUTED claims	112 claims without annotated verifying evidence
Final records	**906 total:** 653 SUPPORTS, 253 REFUTES	**188 total:** 124 SUPPORTS, 64 REFUTES

ClimateFever retained supplied evidence regardless of individual sentence labels. SciFact used annotated sentence indices to extract complete rationale sets. Repeated evidence text was removed, and longer evidence was split into BERT chunks without dropping tokens.

5.4.2 Cross-Domain Performance and Confusion Matrices

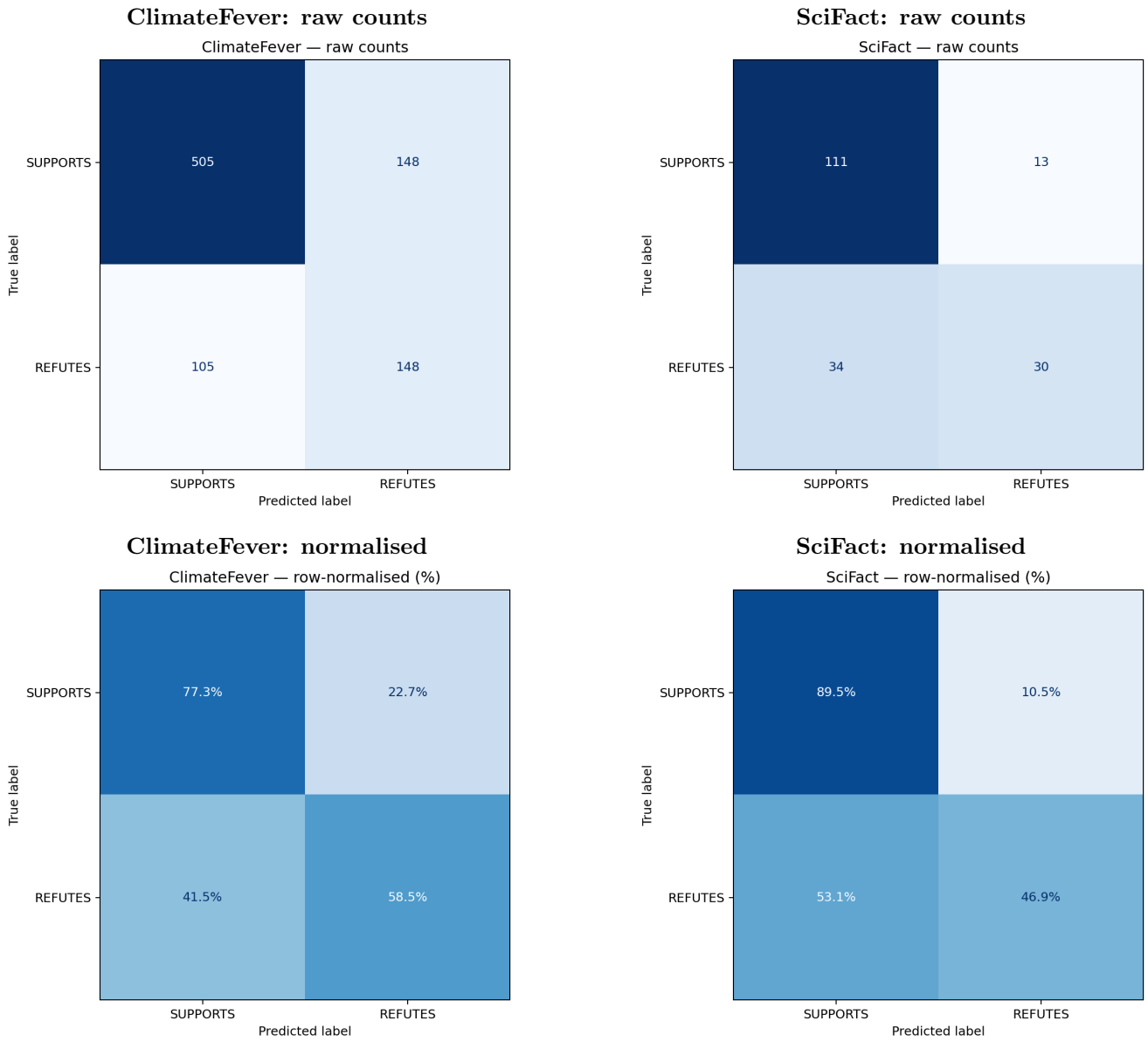


Figure 5.5: Raw-count and row-normalised confusion matrices for ClimateFever and SciFact.

Figure 5.5 shows 253 errors for ClimateFever, consisting of 148 SUPPORTS claims predicted as REFUTES and 105 REFUTES claims predicted as SUPPORTS. SciFact had 47 errors, consisting of 13 SUPPORTS claims predicted as REFUTES and 34 REFUTES claims predicted as SUPPORTS.

Table 5.12: BERT performance on ClimateFever and SciFact

Class	Metric	ClimateFever	SciFact
SUPPORTS	Precision	82.79%	76.55%
SUPPORTS	Recall	77.34%	89.52%
SUPPORTS	F1 score	79.97%	82.53%
REFUTES	Precision	50.00%	69.77%
REFUTES	Recall	58.50%	46.88%
REFUTES	F1 score	53.92%	56.07%
Overall	Accuracy	72.08%	75.00%
Overall	Macro F1 score	66.94%	69.30%

Table 5.12 shows that SciFact achieved higher accuracy and macro F1 score. However, REFUTES recall was lower than on ClimateFever. Both datasets had higher F1 scores for SUPPORTS compared to REFUTES.

5.4.3 Error Patterns and Selected Examples

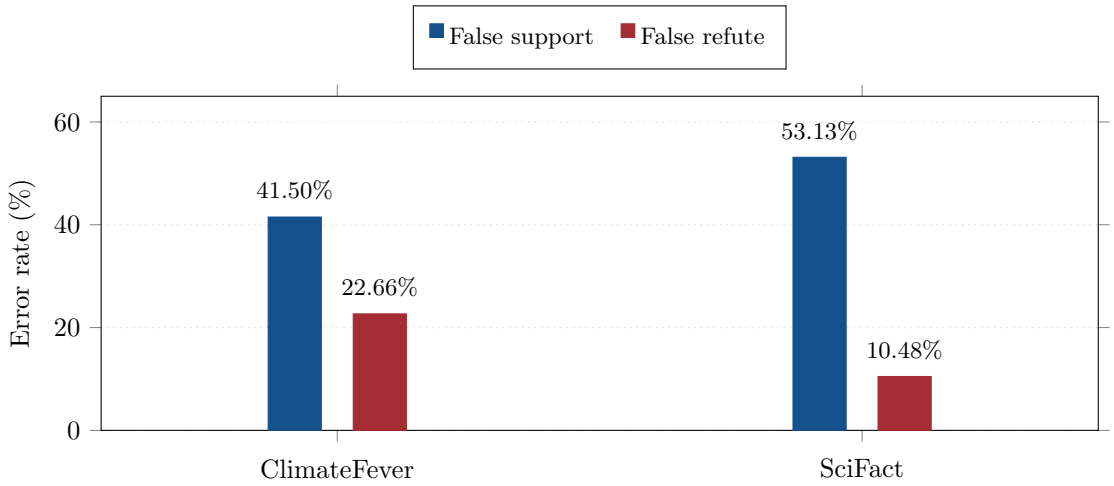


Figure 5.6: False-support and false-refute rates for ClimateFever and SciFact.

False support occurred when REFUTES claims were predicted as SUPPORTS, with rates of 41.50% for ClimateFever and 53.13% for SciFact. False refute occurred when SUPPORTS claims were predicted as REFUTES, with rates of 22.66% for ClimateFever and 10.48% for SciFact. False-support rates were higher in both datasets, suggesting a tendency towards SUPPORTS predictions when classifying REFUTES claims.

For ClimateFever, the claim "The polar bear population has been growing" had an expected label of REFUTES. BERT predicted SUPPORTS with 99.53% confidence. The evidence stated that, of 19 subpopulations, 2 were increasing, 7 were stable, 1 was declining and 9 had insufficient data. Growth in some subpopulations did not establish growth in the overall population.

For SciFact, the claim "ALDH1 expression is associated with better breast cancer outcomes" had an expected label of REFUTES. BERT predicted SUPPORTS with 92.58% confidence. The evidence stated that ALDH1 expression was associated with poor prognosis in a study of 577 breast carcinomas. This contradicted the claim that ALDH1 expression was associated with better outcomes.

Both examples show that high model confidence did not guarantee a correct prediction. The datasets also differed in size, class balance, claim wording and evidence style. These differences may have contributed to the results, so the performance gap could not be attributed to topic alone.

5.5 Application Testing and Explanation Examples

5.5.1 Application Features and Integration Checks

Application checks covered claim verification, alternative input methods, saved results and error handling. Table 5.13 summarises the checks documented during development.

Table 5.13: Application features and checks

Feature	Checks covered
Text verification	Claim submission, passage selection and display of predictions, explanations and evidence
Image/PDF extraction	Text extraction, editable review and rejection of unsupported or oversized files
Speech input and read aloud	Recording, transcription review and playback controls
History and exports	Saved results, reopening conversations and PDF/Word export
Input screening	Rejection of non-claim input without displaying a truthfulness prediction
Error handling	Cancellation, unavailable services and ignored late responses

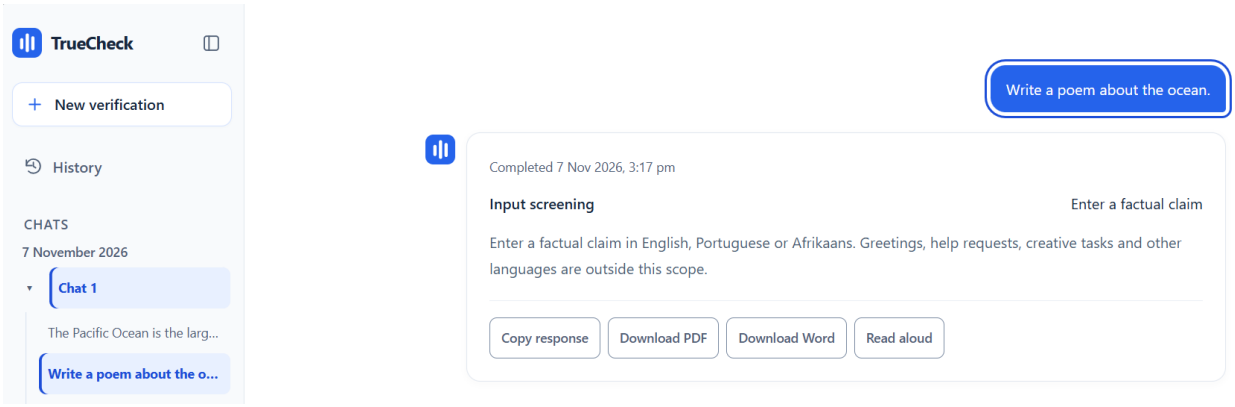


Figure 5.7: Rejection of a creative request through input screening.

The request "Write a poem about the ocean" was rejected because it was not a factual claim. Figure 5.7 shows that the application asked for a factual claim in a supported language and displayed no truthfulness prediction.

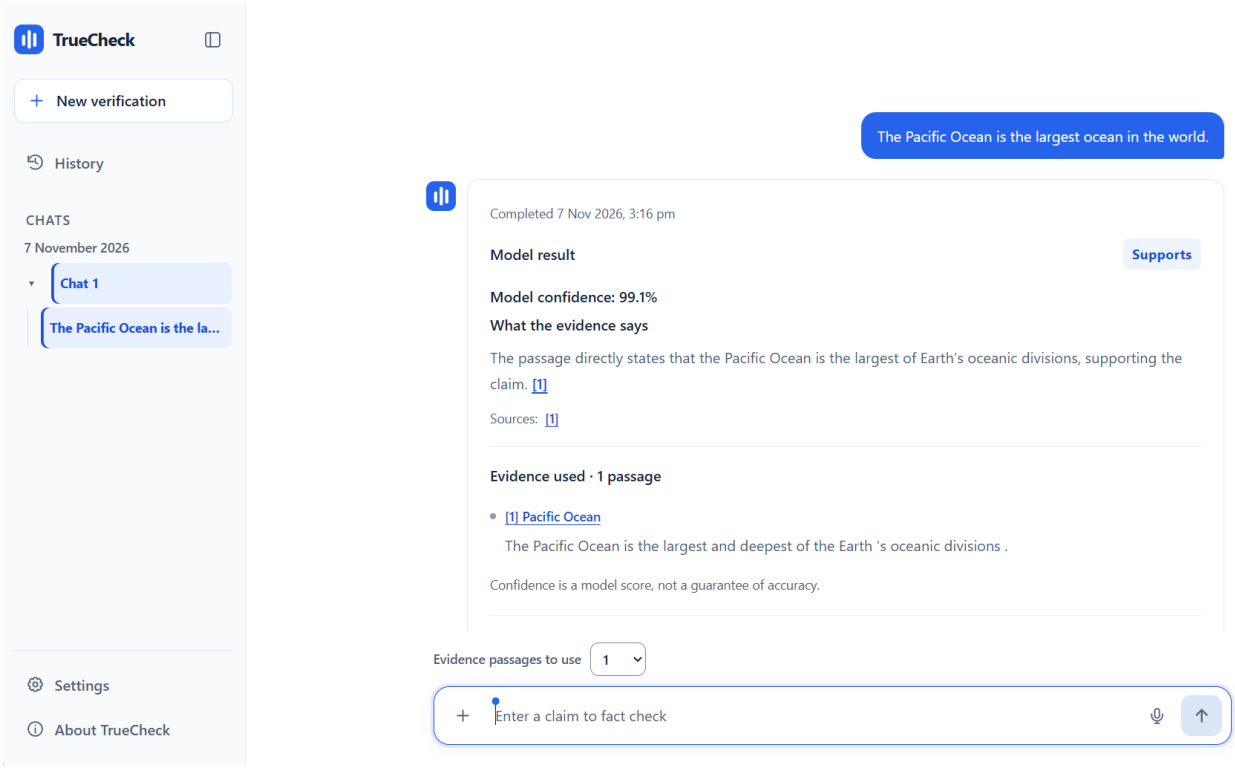


Figure 5.8: English claim verification with retrieved evidence and a cited explanation.

The claim "The Pacific Ocean is the largest ocean in the world" received a SUPPORTS prediction with 99.1% confidence. The retrieved passage stated that the Pacific Ocean was the largest and deepest of Earth's oceanic divisions. The explanation referred directly to this information and cited the displayed source, as shown in Figure 5.8.

This example showed agreement between the claim, retrieved evidence, model prediction and explanation. It was a functional grounding check rather than a measure of explanation quality across many claims.

5.5.2 Multilingual Demonstrations

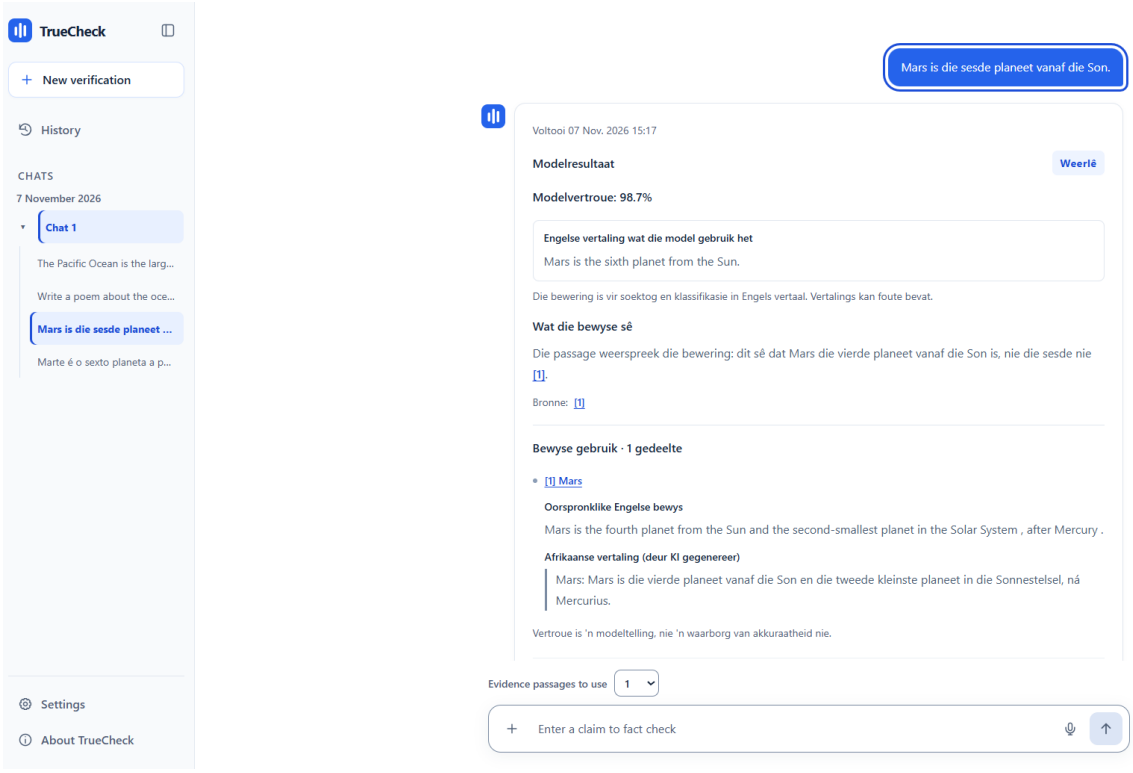


Figure 5.9: Afrikaans claim verification with an English claim translation, a localised explanation and translated evidence.

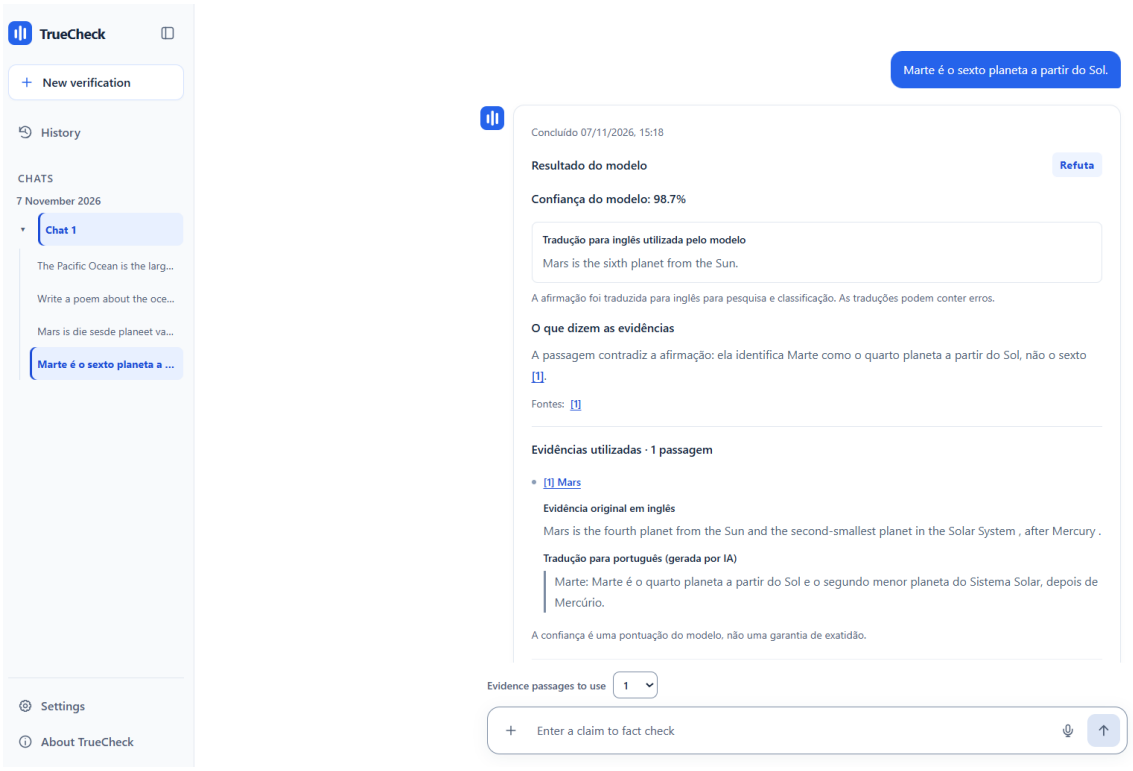


Figure 5.10: Portuguese claim verification with an English claim translation, a localised explanation and translated evidence.

The Afrikaans claim "Mars is die sesde planeet vanaf die Son" and Portuguese claim "Marte é o sexto planeta a partir do Sol" were both translated into "Mars is the sixth planet from the Sun." Both received REFUTES predictions with 98.7% confidence, as shown in Figures 5.9 and 5.10.

The retrieved English evidence identified Mars as the fourth planet from the Sun. Each response explained the contradiction in the original input language and displayed translated evidence alongside the English passage. These examples demonstrated the multilingual workflow.

5.6 Limitations

TrueCheck's BERT classifier was fine-tuned on the prepared FEVER training split, using claims paired with annotated Wikipedia evidence. These claims did not cover the full range of real-world misinformation, including informal language, local claims and breaking news. The retrieval corpus used the same June 2017 Wikipedia snapshot supplied with FEVER, so information about later events or changes could be missing.

The application searched a fixed Wikipedia subset rather than current web sources. Retrieved passages could mention a topic without establishing the claim. The binary classifier still predicted SUPPORTS or REFUTES when passages were returned. Explanations could highlight missing information and flag when the retrieved evidence was insufficient to establish the claim.

The FEVER test scores used annotated evidence and did not establish equivalent performance with automatically retrieved passages. The evidence-count diagnostic provided additional findings, but its internal check split was not an independent final benchmark.

The domain analysis focused on ClimateFever and SciFact. Differences in sample size, class balance, claim wording and evidence sources limited direct comparison. SciFact contained only 188 eligible claims, making class-specific results more sensitive to individual errors. Performance differences could not be attributed to topic alone.

Explanation grounding and multilingual processing were demonstrated through selected examples. Broader testing would be needed to assess citation support, screening reliability and translation accuracy. Model confidence also did not guarantee correctness, as shown by confident errors in the external evaluation.

6. Conclusion

This project developed TrueCheck, a prototype for automated fact verification that combines machine learning classification and evidence-grounded explanation generation. The system demonstrates how these components can be integrated into a single application to help users examine factual claims and understand the evidence behind a classification.

The first question examined how a classifier trained on annotated evidence performs when provided with automatically retrieved passages. The separate retrieval-based diagnostic achieved lower performance than the evaluation using annotated evidence. However, these evaluations used different claims, so the performance gap could not be attributed to retrieval alone. Although the hybrid retriever achieved the best retrieval results among the methods evaluated, retrieving relevant passages did not guarantee accurate classification. In particular, additional passages sometimes introduced information that was related to the claim but did not provide genuine support or refutation. This suggests that evidence relevance and evidence sufficiency are important factors in reliable fact verification.

The second question investigated whether classification performance generalises to claims from other domains. Evaluation on ClimateFever and SciFact showed that the model's performance did not transfer consistently across datasets. The higher false-support rates also highlight a significant limitation: the system can incorrectly present unsupported claims as supported. Consequently, classification performance on the original dataset should not be treated as an indication of reliability across other domains.

The project achieved its main objective of developing a working fact-verification prototype. It implemented and compared classical and transformer-based classifiers, evaluated four evidence retrieval models and integrated a language model to generate explanations grounded in retrieved passages.

Future improvements could include training the classifier on retrieved evidence, updating the evidence corpus with current and local sources, and improving the system's ability to identify insufficient evidence. Further testing could also focus on explanation accuracy, multilingual support and user experience.

TrueCheck demonstrates the potential of automated fact verification through evidence retrieval and machine learning. Although the system has limitations, it provides users with evidence and explanations to help them assess factual claims. It is intended to support, rather than replace, human judgement.

7. How We Used AI Tools

AI tools were used to help with coding, research and report writing. GPT-6.1 Sol with High and Ultra settings in Codex assisted with coding, debugging, dataset preparation and evaluation scripts, and scripts for editing presentations. GPT-6 with High settings in the web interface assisted with research, report drafting and English correction. Generated code and text were manually reviewed before use.

AI generated all 200 claims used in the evidence-count diagnostic, including proposed labels and source links. These were manually reviewed before testing. The diagnostic compared how different numbers of retrieved passages affected BERT's predictions and helped choose the application's default passage count. Its results were reported separately from the held-out FEVER evaluation.

Manual code review identified an incorrect model implementation. AI supplied DistilBERT code even though the selected model was `bert-base-uncased`. The code was corrected to use BERT. This showed that generated code needed to be checked against the chosen model, even when it appeared suitable for the task.

Some generated evaluation scripts also printed too much information and created unnecessary files. For example, a separate `results.md` file stored retrieval results that were later moved into the main README. Removing the file reduced duplication and kept those results in one place. This showed that generated scripts needed to be checked for useful outputs as well as correct calculations.

The AI-generated script used for editing the presentation used an invalid Windows import path and failed with an `ERR_UNSUPPORTED_ESM_URL_SCHEME` error. Changing the import to a valid `file:///` URL allowed the presentation to rebuild successfully.

Classifier performance was checked against labelled data, and selected explanations were compared with the retrieved evidence. Repeated runs produced identical outputs, showing that the process was repeatable. However, this did not mean that every prediction was correct.

AI outputs were reviewed, tested and corrected before being accepted. The team remained responsible for the accuracy of the submitted work, the application's behaviour and the ability to explain the modelling and design decisions.

8. References

Afrobarometer. Promise and peril: Africa's changing media landscape. Afrobarometer Dispatch 509, Afrobarometer, 2022. URL https://www.afrobarometer.org/wp-content/uploads/2022/04/AD509-PAP7-Promise-and-peril-Africas-changing-media-landscape-Afrobarometer-dispatch-19feb22.pdf.

Pepa Atanasova, Jakob Grue Simonsen, Christina Lioma, and Isabelle Augenstein. Generating fact checking explanations. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7352–7364, Online, 2020. Association for Computational Linguistics. URL https://doi.org/10.18653/v1/2020.acl-main.656.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759. Association for Computing Machinery, 2009. doi: 10.1145/1571941.1572114. URL https://doi.org/10.1145/1571941.1572114.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. URL https://doi.org/10.18653/v1/N19-1423.

Thomas Diggelmann, Jordan Boyd-Graber, Jannis Bulian, Massimiliano Ciaramita, and Markus Leippold. CLIMATE-FEVER: A dataset for verification of real-world climate claims. arXiv preprint arXiv:2012.00614, 2020. URL https://arxiv.org/abs/2012.00614.

Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.398. URL https://aclanthology.org/2023.emnlp-main.398/.

Zhijiang Guo, Michael Schlichtkrull, and Andreas Vlachos. A survey on automated fact-checking. *Transactions of the Association for Computational Linguistics*, 10:178–206, 2022. doi: 10.1162/tacl_a_00454. URL https://aclanthology.org/2022.tacl-1.11/.

Institute for Public Policy Research. Countering #elections2024 mis- and disinformation. Technical report, Namibia Fact Check, Institute for Public Policy Research, Windhoek, Namibia, 2025.

Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online, 2020. Association for Computational Linguistics. URL https://doi.org/10.18653/v1/2020.emnlp-main.550.

Nic Newman et al. Digital news report 2025. Technical report, Reuters Institute for the Study of Journalism, 2025. URL https://reutersinstitute.politics.ox.ac.uk/sites/default/files/2025-06/Digital_News-Report_2025.pdf.

Tal Schuster, Darsh Shah, Yun Jie Serene Yeo, Daniel Roberto Filizzola Ortiz, Enrico Santus, and Regina Barzilay. Towards debiasing fact verification models. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3419–3425. Association for Computational Linguistics, 2019. doi: 10.18653/v1/D19-1341. URL https://aclanthology.org/D19-1341/.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: a large-scale dataset for fact extraction and VERification. In *Proceedings of NAACL-HLT 2018, Volume 1 (Long Papers)*, pages 809–819. Association for Computational Linguistics, 2018. doi: 10.18653/v1/N18-1074. URL https://aclanthology.org/N18-1074/.

David Wadden, Shanchuan Lin, Kyle Lo, Lucy Lu Wang, Madeleine van Zuylen, Arman Cohan, and Hannaneh Hajishirzi. Fact or fiction: Verifying scientific claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7534–7550. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.emnlp-main.609. URL https://aclanthology.org/2020.emnlp-main.609/.

Appendix A: Proposed Project Schedule and Features

A.1 Proposed Project Schedule

Table A.1: Proposed project schedule

Week	Dates	Main Focus
Week 1	Mon 7 - Fri 11 Sep	Understand TrueCheck and the FEVER dataset. Inspect the claims, labels and evidence structure, agree on how the system should work, discuss suitable Classical and Transformer models, and set up GitHub/project structure.
Week 2	Mon 14 - Fri 18 Sep	Prepare and preprocess the dataset, explore class distribution, create train/validation/test splits, define evaluation metrics and prepare the modelling pipelines.
Week 3	Mon 21 - Fri 25 Sep	Split into 2 people on the Classical model and 2 people on the Transformer model. Train and compare both approaches and record the initial results.
Week 4	Mon 28 Sep - Fri 2 Oct	Finish training and comparing the Classical and Transformer models and select the model to use in the final pipeline. Prepare the mock presentation. Begin building the evidence corpus, similarity search and retrieval component. Mock Project Presentation - Wed 30 Sep.
Week 5	Mon 5 - Fri 9 Oct	Complete evidence retrieval and ranking. Connect the selected classifier to the retrieval pipeline. Add claim detection, input validation, gibberish detection, out-of-scope handling, ambiguous-claim handling and insufficient-evidence handling. Begin checking errors and bias across different claim types.
Week 6	Mon 12 - Fri 16 Oct	Build the LLM + RAG component. Generate evidence-grounded explanations and citations, add guardrails and test groundedness. Begin web application development now that the main NLP components exist.
Week 7	Mon 19 - Fri 23 Oct	Continue the web application and full-system integration. Add OCR/image input, OCR confirmation, URL/article input, file upload, history/dashboard and shareable results. Continue bias/error analysis and multilingual testing.
Week 8	Mon 26 - Fri 30 Oct	Add voice input, voice response, speech confirmation, multilingual voice/text handling, accent-aware speech recognition and error/API-failure handling. Complete integration, deploy the web app, test the full system and fix bugs. Prepare the slides, 2-minute video and final demo. Final Project Presentation + Demo - Wed 28 Oct.
Week 9	Mon 2 - Fri 6 Nov	Finish the report, model comparison, evaluation, bias/error analysis, multilingual discussion, AI-use reflection, screenshots and README. Clean up GitHub/code and prepare the final submission. Project Report + Code Due - Fri 6 Nov.

A.2 Proposed Features

Table A.2: Proposed application features

Feature	Brief Explanation
Voice Input / Speech-to-Text	Let users speak a claim instead of typing it.
Voice Response / Text-to-Speech	Read the verdict, explanation and evidence aloud.
OCR from Images	Extract claims from screenshots, posters, memes or news images.
URL / Article Input	Let users paste an article or news link for checking.
Web Application + Deployment	Provide a browser-based TrueCheck interface and deploy it online.
File Upload	Allow PDFs, images and text files to be uploaded for checking.
Shareable Results	Let users copy or share a verification result and its evidence.
History / Dashboard	Let users view previously checked claims and results.
Multilingual Input	Support claims in other languages where possible.

Feature	Brief Explanation
Accent-Aware Speech Recognition	Improve speech recognition for different accents, including Afrikaans-accented English.
Input Validation	Detect empty, invalid, too short or too long input.
Gibberish Detection	Detect random or meaningless text before verification.
Claim Detection	Determine whether the input is actually a factual claim that can be verified.
Out-of-Scope Handling	Reject requests that are unrelated to fact-checking.
Ambiguous Claim Handling	Ask for clarification when a claim lacks enough context.
Insufficient Evidence Handling	Return Unverified / Not Enough Evidence rather than guessing.
OCR / Speech Confirmation	Let the user correct extracted or transcribed text before verification.
Error Handling	Give useful messages when the model, OCR, retrieval system, speech service or API fails.